

AquaGuard

Documento de requisitos, arquitectura y planificación

ft_transcendence - Plataforma web de monitorización de instalaciones internas de agua

Objetivo de esta V3.1

Unificar el subject de *ft_transcendence*, el producto que queremos crear, el planning original del equipo y la revisión técnica posterior, fácil de entender y presentar. La V3.0 conserva las decisiones útiles del planning inicial (Vue, dashboard admin/cliente, mocks, componentes compartidos, mapa y trabajo paralelo) y añade las decisiones de arquitectura necesarias, en esta version se agrega el integrante 5 para el proyecto.

Dato	Decisión V2
Arquitectura	Monolito modular
Equipo	5 personas: 2 Frontend + 2 Backend/Data + 1 Integration/Infra/DevOps
Stack	Vue 3 + FastAPI + PostgreSQL
Infraestructura	Docker Compose + Nginx
Ejecución final	Makefile
Objetivo temporal	Máximo 6 semanas
Prioridad	1) Mandatory, 2) 14 puntos de módulos, 3) Bonus

Base documental: subject *ft_transcendence* v2.1 + AquaGuard Planning Frontend v1.2 + Planning técnico revisado.

Índice de trabajo

1. Visión, alcance y principios
- 2. Requisitos del subject (Mandatory)**
- 3. Arquitectura: monolito modular**
4. Stack tecnológico
5. Modelo de dominio y base de datos
- 6. Docker, contenedores y Nginx**
7. Especificaciones Frontend
8. Especificaciones Backend
9. API: endpoints por nivel
10. Comunicación Frontend-Backend y mocks
11. Estructura del repositorio
12. Testing y calidad
- 13. Equipo, roles y tareas**
14. Git y gestión de ramas
- 15. Plan de módulos: 14 puntos**
16. Bonus recomendados
- 17. Plan máximo de 6 semanas**

1. Visión, alcance y principios

1.1 Producto

AquaGuard es una **plataforma web** para la **monitorización** de **instalaciones internas** de **agua** en hoteles y edificios **mediante sensores**.

Permite supervisar mediciones como la presión del agua, consultar históricos y generar alertas cuando se detectan valores anómalos o sensores sin conexión.

Durante el desarrollo no se utilizarán inicialmente sensores físicos. Un **simulador en Python** generará lecturas ficticias que se **comportan** como **datos procedentes** de **sensores reales**.

El objetivo del MVP no será diagnosticar automáticamente una fuga, sino proporcionar una herramienta de **monitorización y detección de anomalías** que ayude a administradores y responsables de mantenimiento a identificar posibles problemas dentro del edificio.

1.2 Caso base del MVP

- Una aplicación web multiusuarios con perfiles **Admin** y **Cliente**.
- Entre **1 y 3 edificios/edificios** de demostración.
- Varios sensores simulados distribuidos en diferentes zonas del edificio.
- Medición principal: **presión del agua (bar)**, opcionalmente caudal (L/min).
- **Alertas iniciales:**
 - presión baja
 - presión alta,
 - sensor sin datos/ desconectado.
- **Dashboard Admin** con visión global, mapa, KPIs, edificios, sensores y alertas.
- **Dashboard Cliente** limitado a sus instalaciones, sensores, históricos y alertas.
- Consulta del histórico de mediciones mediante gráficas.

1.3 Principios de diseño

- Primero completar el **flujo vertical**:
 - Simulador(sensores) -> API -> PostgreSQL -> API -> Vue.
- Utilizar un **monolito modular**: una única aplicación **FastAPI** organizada por **módulos** de **negocio**.
- Mantener una única aplicación **Vue 3 SPA(single page application)**, dividida en Zona Pública, Administración y Cliente.
- El frontend nunca accederá directamente a PostgreSQL; toda comunicación se realizará mediante **FastAPI**.
- Implementar primero la comunicación mediante **REST**: autenticación, sensores, lecturas, históricos y CRUD.
- Añadir **WebSockets** cuando el flujo REST sea estable, principalmente para nuevas lecturas y alertas en tiempo real.
- Mantener estable el **contrato JSON** entre frontend y backend para que los mocks y la API real tengan la misma estructura.
- El **simulador solo genera mediciones**. La detección de anomalías y creación de alertas corresponde a la lógica de negocio del backend.
- Mantener una arquitectura sencilla ..
- Priorizar en este orden: **Mandatory** → **14 puntos de módulos** → **Bonus**.
- No comenzar funcionalidades bonus hasta que el Mandatory y los módulos necesarios sean funcionales y demostrables.

2. Requisitos del subject (Mandatory)

Objetivo global proyecto y del equipo.

Requisito	Qué exige	Aplicación en AquaGuard
Aplicación web	Frontend, backend y base de datos.	Vue + FastAPI + PostgreSQL
Git	Commits de todos, mensajes claros y reparto visible.	PRs y ramas por integrante e issue desde develop
Contenedores	Ejecutar utilizando makefile.	docker compose up --build
Navegador	Compatible con la última versión estable de Chrome .	Prueba final en Chrome
Consola de navegador	Sin warnings ni errores en consola.	Criterio de Definition of Done
Legal	Privacy Policy , Terms of Service accesibles y con contenido real.	Footer público y dentro de la app
Multiusuario	Varios usuarios conectados simultáneamente sin corrupción/conflictos.	Sesiones separadas, DB transaccional y pruebas concurrentes
Frontend	Claro, responsive y accesible en dispositivos .	Tailwind/DaisyUI + revisión móvil/tablet/desktop
CSS	Usar framework o solución de estilos.	Tailwind CSS + DaisyUI
Secrets	Variables en .env ignorado por Git y .env.example .	.gitignore + plantilla
DataBase	Esquema claro y relaciones definidas.	SQLAlchemy + diagrama + Alembic
Usuarios	Registro y login con email/password seguros.	Hash Argon2/bcrypt y sesión/token seguro
Validación	Validacion en frontend y backend .	Vue + Pydantic
HTTPS	Toda conexión externa al backend debe usar HTTPS .	Nginx termina TLS
README	README en inglés.	Se mantiene desde semana 1

2.1 README obligatorio

- Primera línea en cursiva con los logins del equipo según el texto del subject.
- Description: nombre, objetivo, overview y features principales.
- Instructions: prerequisites, versiones, .env y pasos para ejecutar.
- Resources: documentación/referencias y explicación de cómo se utilizó IA.
- Team Information: roles y responsabilidades.
- Project Management: distribución, reuniones, herramientas y canales.
- Technical Stack: frontend, backend, DB y justificación.
- Database Schema: tablas, relaciones, campos y tipos clave.
- Features List: funcionalidad y miembros que participaron.
- Modules: Major/Minor, puntos, justificación, implementación y participantes.
- Individual Contributions: desglose, retos y soluciones.

Regla de evaluación

Todos los integrantes deben poder explicar la arquitectura, su contribución y realizar cambios pequeños durante la evaluación. La documentación no sustituye al conocimiento del código.

3. Arquitectura del sistema

AquaGuard seguirá una arquitectura sencilla basada en dos bloques principales:

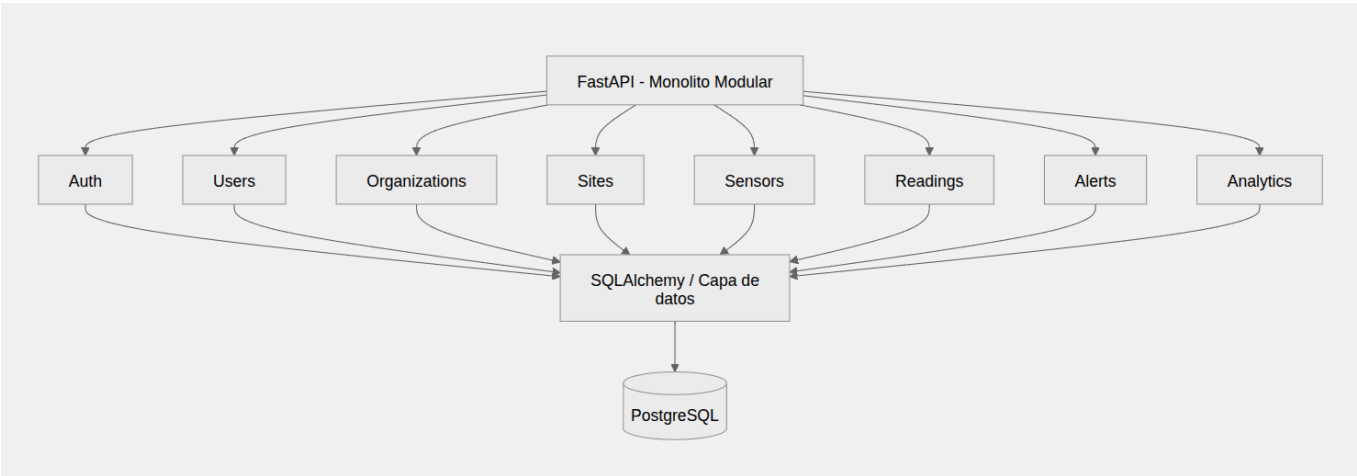
- **Backend:** monolito modular desarrollado con Fast API.
- **Frontend:** SPA modular desarrollada con Vue 3.

El objetivo es separar responsabilidades sin introducir la complejidad de una arquitectura de microservicios.

3.1 Arquitectura Backend - Monolito Modular

AquaGuard tendrá un **único backend desplegable**, estará dividido en **módulos de negocio**.

Esto permite mantener **separadas** las **responsabilidades** y **evita concentrar** toda la **lógica** en un **archivo**.



Aunque estén separados internamente, **todos los módulos pertenecen a una única aplicación Fast API y a un único contenedor backend**.

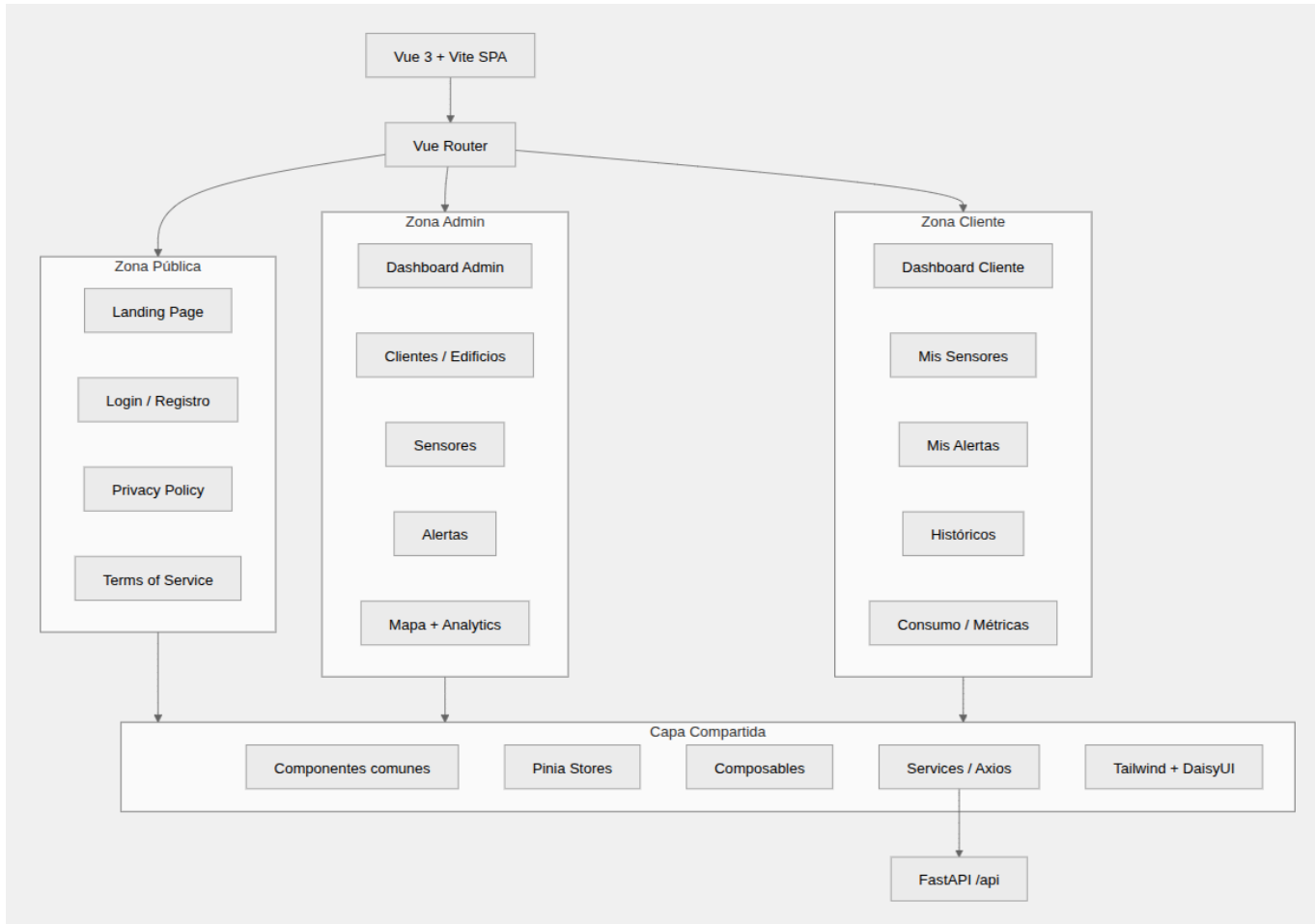
Módulo	Responsabilidad
auth	Registro, login, sesión/token, permisos básicos.
users	Perfil y administración de usuarios cuando aplique.
organizations	Empresa/cliente y miembros; (necesario si se reclama Organization System).
sites	Hoteles/edificios y localización.
sensors	Sensores, configuración, umbrales y estado.
readings	Recepción y consulta de mediciones.
alerts	Reglas, reconocimiento y resolución.
analytics	Agregaciones, KPIs, filtros por fechas y exportación.

3.2 Arquitectura Frontend - SPA modular

El frontend será una única aplicación **Vue 3 SPA**, dividida principalmente en tres áreas:

- Landbase
- Administración.
- Cliente.

Public, **Admin** y **Cliente** compartirán componentes, estilos, servicios y estado para evitar duplicar código.



Módulo	Responsabilidad
public	Landing, login, registro, Privacy Policy y Terms of Service.
admin	Dashboard global, clientes, edificios, sensores, mapa y alertas.
client	Dashboard propio, sensores, alertas e históricos del cliente.
components	Cards, tablas, formularios, modales, Header, Footer, etc.
stores	Estado global con Pinia: usuario, sesión, sensores y alertas.
composables	Lógica reutilizable entre componentes.
services	Comunicación con el backend y organización de llamadas API.
router	Navegación y protección de rutas según autenticación/rol.

4. Stack tecnológico

Área	Tecnología	Uso
Lenguaje Frontend	JavaScript (opcional TypeScript)	Código Vue y lógica de interfaz.
Framework Frontend	Vue 3 + Vite	SPA, componentes y desarrollo rápido.
Estado	Pinia	Sesión, usuario, sensores, alertas y notificaciones.
UI	Tailwind CSS + DaisyUI	Responsive y componentes reutilizables.
Mapas	Leaflet + OpenStreetMap	Mapa de edificios/sensores.
Gráficas	Chart.js	Histórico y analítica.
Cliente HTTP	Axios	REST, interceptores y errores.
Lenguaje Backend	Python 3	API, simulador y scripts.
Framework Backend	FastAPI + Pydantic	REST, validación, OpenAPI y WebSockets.
ORM	SQLAlchemy	Acceso a datos.
Migraciones	Alembic	Evolución reproducible del esquema.
Base de datos	PostgreSQL	Persistencia relacional.
Proxy	Nginx	HTTPS, SPA y reverse proxy.
Contenedores	Docker Compose	Entorno reproducible con un comando.
Testing	Pytest + Vitest + Playwright	Unitario, integración y E2E.

Decisión FastAPI

Como todo el equipo es nuevo en el stack, FastAPI reduce la configuración manual gracias a Pydantic y OpenAPI/Swagger. La parte que requerirá más aprendizaje será SQLAlchemy/Alembic y el diseño correcto de relaciones.

5. Modelo de dominio y base de datos

Se propone una base de datos de complejidad básica-media: suficiente para usuarios, organizaciones, edificios, sensores, lecturas y alertas.

```
Organization                                Hotel Group
|-- User                                  |-- Usuario: admin@hotel.com
|-- Site (hotel / edificio)              |-- Hotel: Barcelona
    |-- Sensor                          |-- Sensor cocina
        |-- Reading                      |-- Sensor planta 1
        |-- Alert                       |-- Sensor planta 2
```

Tabla	Campos esenciales	Relación / nota
organizations	Id, name, created_at	Empresa/cliente.
users	id, organization_id, name, email, password_hash, role, created_at	Cuenta de acceso.(ADMIN CLIENT)
sites	id, organization_id, name, address, latitude, longitude, created_at	Hotel/edificio
sensors	id, site_id, name, location, sensor_type, min_pressure, max_pressure, status, last_seen_at, created_at	Sensor físico/simulado.
readings	id, sensor_id, pressure, created_at	Sensor simulado.
alerts	id, sensor_id, type, severity, message, status, created_at, resolved_at	Incidencia generada por reglas: LOW_PRESSURE, HIGH_PRESSURE SENSOR_OFFLINE Estado: ACTIVE , RESOLVED Severidad: WARNING, CRITICAL

5.1 Reglas de base de datos

- No duplicar datos derivados como “última lectura” en Site; se consulta Reading.
- Índice en readings (sensor_id, measured_at).
- Email único y código de sensor único dentro de su ámbito.
- Migraciones Alembic revisadas por ambos backend.
- Seed pequeño para desarrollo; dataset de 100+ sitios/sensores solo para pruebas de paginación/carga.
- PostgreSQL utiliza un volumen persistente de Docker.

6. Docker, contenedores y Nginx

Se recomiendan 4 contenedores.

Contenedor	Imagen	Qué contiene / guarda
gateway	Imagen propia multi-stage: Node build + Nginx	Vue compilado (dist), configuración Nginx y certificados de evaluación. No guarda datos de negocio.
backend	Imagen propia Python	FastAPI, Pydantic, SQLAlchemy, Alembic y código del monolito modular.
database	postgres oficial	Datos persistentes en volumen: usuarios, sitios, sensores, lecturas, alertas.
simulator	Imagen propia Python	Generador de lecturas y escenarios de prueba; llama a la API, no toca la DB directamente.

Contenedor opcional de desarrollo

Puede existir un servicio frontend-dev con Vite y HMR mediante un profile de Docker Compose. No es necesario en la topología final y no debe complicar la evaluación.

6.1 Redes y exposición

- Solo gateway expone puertos al host: 80 y 443.
- Backend (8000) y PostgreSQL (5432) permanecen en la red interna de Compose.
- Simulator llama al backend por el nombre de servicio interno.
- PostgreSQL usa volumen persistente.
- Health checks mínimos: backend y database.
- El comando final será docker compose up --build.

6.2 Función de Nginx

Nginx será el **punto de entrada** de toda la app. Sirve Vue, fuerza HTTPS y decide si una petición debe quedarse en el frontend o enviarse a FastAPI.

```
server {
    listen 80;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    root /usr/share/nginx/html;
    location / {
        try_files $uri $uri/ /index.html;
    }
    location /api/ {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
    }
    location /ws/ {
        proxy_pass http://backend:8000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}
```

Nota: los certificados y rutas exactas se cerrarán al configurar el entorno de evaluación. El objetivo de la V2 es fijar la responsabilidad del proxy, no congelar todavía cada directiva.

7. Especificaciones Frontend

El frontend de AquaGuard será una única aplicación **Vue 3 SPA + Vite**, dividida en tres áreas principales:

- **Zona Pública:** Landing, Login, Registro, Privacy Policy y Terms of Service.
- **Zona Admin:** gestión global de clientes, sites, sensores, alertas y monitorización.
- **Zona Cliente:** información y monitorización correspondiente al cliente autenticado.

Las tres áreas compartirán **componentes, estilos, stores, servicios** y comunicación con la API.

7.1 Division de responsabilidades

Area	Florinda	Lylia
Zona Pública	Landing Page, diseño general, Privacy y Terms	Login, Registro, validaciones e integración Auth (integración con Backend)
Zona Admin	Dashboard Admin, KPIs, mapa Leaflet, clientes/sites	Sensores, alertas, tablas, filtros e integración API (integración con Backend)
Zona Cliente	Apoyo visual y componentes compartidos	Dashboard Cliente, sensores, alertas, históricos y gráficas
Componentes comunes	Layout, Header, Sidebar, Cards, Modal y sistema visual	Loading, Error, EmptyState, tablas y formularios reutilizables
Estado / Datos	Colaboración y definición de necesidades	Pinia, Axios, Services y Composables
Tiempo real	Representación visual de cambios	WebSocket, actualización de Stores (Backend)
Testing	Responsive, UX y accesibilidad	Vitest, E2E e integración

7.2 Reglas Frontend

Florinda - Frontend Lead / Admin

- Definir estructura y arquitectura visual de Vue.
- Desarrollar Landing Page.
- Crear layouts y navegación general.
- Crear componentes visuales base.
- Desarrollar Dashboard Admin y KPIs.
- Integrar mapa Leaflet.
- Desarrollar gestión visual de clientes/sites.
- Supervisar consistencia visual, responsive y accesibilidad.

Lylia - Frontend Client / Integration

- Configurar Pinia y stores.
- Configurar Axios y capa **services**.
- Implementar Login/Registro y sesión (**Backend**).
- Desarrollar sensores y alertas del Admin.
- Desarrollar Dashboard Cliente completo.
- Implementar históricos y Chart.js.
- Integrar endpoints REST (**Backend**).
- Implementar WebSockets y testing E2E (**Backend**).

7.3 Prioridades de desarrollo

Prioridad	Florinda	Lylia	Resultado
1. Setup	Vue, Vite, Router, Tailwind, DaisyUI, layouts	Pinia, Axios, Services, mocks	Base frontend
2. Vertical Slice	SensorCard + vista sensor	Consumir Sensors/Readings (Backend)	Datos del simulador visibles en Vue
3. Público/Auth	Landing, Privacy, Terms	Login, Registro, Auth Store (Backend)	Acceso funcional
4. Admin	Dashboard, KPIs, mapa, clientes/sites	Sensores, alertas, tablas y filtros (Backend)	Administración funcional
5. Cliente	Componentes/apoyo visual	Dashboard, sensores, alertas, históricos	Área Cliente funcional
6. Analytics	KPIs y gráficas Admin	Chart.js, fechas y exportación (Backend)	Monitorización completa
7. Real-time	UI de alertas/cambios	WebSocket + Stores (Backend)	Actualización en tiempo real
8. Calidad	Responsive, UX, accesibilidad	E2E, errores y bugs	Frontend preparado para evaluación

8. Especificaciones Backend

El backend de AquaGuard será un **monolito modular desarrollado con FastAPI**.
Cada módulo de negocio tendrá una estructura interna similar:



Esto permite separar la lógica HTTP, la lógica de negocio y el acceso a datos sin introducir la complejidad de microservicios.

8.1 Estructura interna de cada módulo

Capa	Responsabilidad
router.py	Define endpoints, recibe peticiones y devuelve respuestas HTTP.
service.py	Contiene la lógica de negocio y coordina operaciones.
repository.py	Centraliza consultas y operaciones contra PostgreSQL.
model.py	Define las tablas y relaciones con SQLAlchemy.
schemas.py	Define los datos de entrada/salida con Pydantic.

POST /api/readings -> readings/[router.py](#) -> readings/[service.py](#) -> readings/[repository.py](#) -> Reading Model -> PostgreSQL.

8.2 Módulos Backend

El backend se organizará por **módulos de negocio**, y cada módulo contendrá sus propias capas:

```
modules/  
├─ auth/  
├─ users/  
├─ organizations/  
├─ sites/  
├─ sensors/  
├─ readings/  
├─ alerts/  
└─ analytics/
```

Cada **módulo** seguirá, cuando aplique:

```
router.py    → endpoints HTTP  
service.py   → lógica de negocio  
repository.py → acceso a datos  
model.py     → modelo SQLAlchemy  
schemas.py   → validación Pydantic
```

Módulo	Responsabilidad
auth	Registro, login, sesión/token y permisos.
users	Perfil y gestión de usuarios.
organizations	Empresas/clientes y miembros.
sites	Hoteles/edificios y localización.
sensors	Sensores, estado y umbrales.
readings	Recepción y consulta de mediciones.
alerts	Creación y resolución de alertas.
analytics	KPIs, históricos, filtros y exportación.

8.3 Responsabilidades del equipo Backend

Ana — Backend API & Business Logic

Responsable principalmente de **FastAPI** y **lógica de negocio**:

- Crear proyecto FastAPI y estructura modular.
- Configurar `main.py`, routers y dependencias.
- Crear contratos **Pydantic**.
- Implementar endpoints **REST**.
- Implementar **autenticación y permisos**.
- Implementar lógica de sensores y lecturas.
- Implementar reglas de alertas.
- Configurar Swagger/OpenAPI.
- Implementar **WebSockets** cuando **REST** sea **estable**.
- Crear tests de servicios y endpoints.

Daruny — Backend Data & Infrastructure

Responsable principalmente de **persistencia** e **infraestructura**:

- Configurar PostgreSQL.
- Configurar SQLAlchemy.
- Crear modelos y relaciones.
- Configurar Alembic y migraciones.
- Crear repositories.
- Crear seeders y datos de prueba.
- Crear simulador Python de sensores.
- Crear consultas y filtros para históricos.
- Configurar Docker del backend y database.
- Gestionar índices, health checks y backups básicos.

8.4 Prioridades de desarrollo Backend

Prioridad	Ana	Daruny	Objetivo
1. Base técnica	FastAPI, estructura modular, routers, Pydantic, errores	PostgreSQL, SQLAlchemy, Alembic, modelos iniciales	FastAPI conectado a PostgreSQL
2. Vertical Slice	POST /readings, GET /sensors, históricos	Simulator, repositories, seed de sensores	Simulator → API → DB → Vue
3. Auth	Registro, login, JWT/sesión, protección rutas	User, Organization, relaciones y seeds	Multiusuario funcional
4. Alertas	Reglas LOW/HIGH, endpoints, resolución	Alert model, consultas y simulación de errores	Alertas automáticas
5. Sites/Sensores	CRUD y permisos	Relaciones, constraints, paginación	Gestión Admin
6. Analytics	Endpoints de métricas y exportación	Agregaciones y consultas SQLAlchemy	Dashboards y gráficas
7. Real-time	WebSockets y broadcasting	Simulator + pruebas de concurrencia	Actualización en tiempo real
8. Calidad	Tests, errores, documentación API	Docker, health checks, backups	Backend preparado para evaluación

Importante: Responsabilidad transversal de Eduardo

Durante las prioridades 1-8, Eduardo acompaña la integración: Docker Compose y Nginx desde la base técnica; Pytest/Playwright, health checks y smoke tests durante calidad y release.

8.5 Primera funcionalidad obligatoria

El primer objetivo del equipo Backend será completar este flujo:

Simulator Python
↓
POST /api/readings
↓
FastAPI
↓
Service
↓
Repository
↓
SQLAlchemy -> PostgreSQL

Después:

Vue
↓
GET /api/sensors/{id}/readings
↓
FastAPI
↓
PostgreSQL
↓
JSON
↓
Vue

Hasta que este flujo funcione correctamente, no se priorizan funcionalidades más complejas como WebSockets o analytics avanzados.

9. API: endpoints por nivel

La API se divide en tres niveles. El equipo debe terminar “Básico” antes de ampliar “Intermedio”; “Full” solo se implementa si aporta a módulos/bonus.

9.1 Básico - Mandatory + MVP

```

POST    /api/auth/register
POST    /api/auth/login
POST    /api/auth/logout          (o invalidación de sesión)
GET     /api/me

GET     /api/sites
GET     /api/sites/{id}
GET     /api/sites/{id}/sensors

GET     /api/sensors
POST    /api/sensors
GET     /api/sensors/{id}
PATCH  /api/sensors/{id}

POST    /api/readings
GET     /api/sensors/{id}/readings

GET     /api/alerts
PATCH  /api/alerts/{id}/acknowledge
PATCH  /api/alerts/{id}/resolve

GET     /api/health

```

POST /api/readings será especialmente importante porque lo utilizará el **simulador**

9.2 Intermedio - plan de 14 puntos

```

GET/POST/PATCH/DELETE /api/users/...
GET/POST/PATCH/DELETE /api/organizations/...

POST    /api/organizations/{id}/members
DELETE  /api/organizations/{id}/members/{user_id}

POST    /api/sites
PATCH  /api/sites/{id}
DELETE  /api/sites/{id}

DELETE  /api/sensors/{id}

GET     /api/analytics/overview?from=&to=
GET     /api/analytics/sensors/{id}?from=&to=

GET     /api/alerts?severity=&status=&page=&sort=
GET     /api/sites?search=&page=&sort=
GET     /api/sensors?site_id=&status=&page=

GET     /api/export/readings.csv

GET     /ws/monitoring
GET     /api/status

```

Aquí ya entran **CRUD completos**, filtros, analytics y tiempo real.

9.3 Full - solo si hay tiempo

```

POST /api/public/readings      (API key + rate limit si se reclama Public API)
GET  /api/public/sites        (API pública documentada)
POST /api/notifications/test   (canales externos)
POST /api/chat/conversations   (solo si se decide implementar interacción)
POST /api/chat/messages
GET  /api/health
GET  /api/status

```

10. Comunicación Frontend-Backend y mocks

```

Vue View/Component
  |
  v
Pinia / Composable
  |
  v
ApiService (interfaz única)
  |
+-----> MockAdapter (inicio)
  |
+-----> HttpAdapter / Axios (API real)
          | HTTPS / JSON
          v
          Nginx
          v
          FastAPI

```

- OpenAPI es la fuente de verdad para nombres de campos, tipos, códigos HTTP y paginación.
- MockAdapter y HttpAdapter deben devolver el mismo shape.
- Los cambios incompatibles del contrato requieren issue y revisión de un frontend y un backend.
- Frontend no espera hasta semana 4 para integrar: al final de semana 1 debe existir al menos un flujo real end-to-end.
- Formato de error acordado, por ejemplo: {"error": {"code": "...", "message": "...", "details": ...}}.

11. Estructura del repositorio

```

aquaguard/
|-- README.md
|-- compose.yaml
|-- .env.example
|-- .gitignore
|-- Makefile                                # comandos comunes del proyecto
|
|-- frontend/
|   |-- src/
|   |   |-- components/                    # UI compartida
|   |   |-- views/
|   |   |   |-- public/                    # Landing, login, registro, legal
|   |   |   |-- admin/                     # Dashboard y gestión Admin
|   |   |   |-- client/                     # Dashboard y vistas Cliente
|   |   |-- layouts/
|   |   |-- router/
|   |   |-- stores/                         # Pinia
|   |   |-- services/                       # Axios / API
|   |   |-- composables/
|   |   |-- utils/
|   |-- tests/
|   |-- package.json
|   |-- Dockerfile                          # desarrollo / build cuando aplique
|
|-- backend/
|   |-- app/
|   |   |-- main.py
|   |   |-- core/
|   |   |   |-- config.py
|   |   |   |-- database.py
|   |   |   |-- security.py
|   |   |   |-- exceptions.py
|   |   |-- modules/
|   |   |   |-- auth/
|   |   |   |-- users/
|   |   |   |-- organizations/
|   |   |   |-- sites/
|   |   |   |-- sensors/
|   |   |   |-- readings/
|   |   |   |-- alerts/
|   |   |   |-- analytics/
|   |   |   |   # Cada modulo usa, cuando aplique:
|   |   |   |   # router.py / service.py / repository.py / model.py / schemas.py
|   |   |-- shared/
|   |   |   |-- dependencies.py
|   |   |   |-- utils.py
|   |-- migrations/                         # Alembic
|   |-- seeds/                             # datos iniciales / demo
|   |-- tests/
|   |   |-- unit/
|   |   |-- integration/
|   |-- requirements.txt                     # o pyproject.toml
|   |-- Dockerfile
|
|-- simulator/
|   |-- app/
|   |   |-- main.py
|   |   |-- scenarios.py                    # normal, low/high pressure, offline
|   |-- Dockerfile
|
|-- gateway/
|   |-- Dockerfile                          # build Vue + Nginx runtime
|   |-- nginx.conf
|   |-- certs/                             # sin secretos reales en Git
|
|-- docs/
|   |-- decisions/
|
|-- .github/
|   |-- workflows/
|   |   |-- ci.yml                          # lint, tests y build en Pull Request
|   |   |-- docker.yml                     # smoke/build Docker si se decide separar

```

12. Testing y calidad

El subject no exige una cifra concreta de tests, pero la concurrencia, el login seguro y la capacidad de explicar el proyecto hacen recomendable una estrategia pequeña y útil.

Tipo	Herramienta	Qué probamos
Backend unit/integration	Pytest	Auth, permisos, CRUD, reglas de alertas, filtros y conflictos.
Frontend unit	Vitest	Stores, validadores, composables y componentes críticos.
E2E	Playwright	Registro/login, navegación por rol, lectura de sensor, gestión de alerta, logout.
Concurrencia básica	Playwright/API tests	Dos sesiones simultáneas realizando acciones sin mezclar datos.
Infraestructura	Docker healthchecks + smoke test	Gateway, backend, DB y flujo simulador -> API.

12.1 Alcance recomendado

- No buscar 100% de coverage. Priorizar rutas críticas.
- Ana y Daruny implementan pruebas de sus endpoints/reglas.
- Florinda y Lylia implementan pruebas de su flujo.
- Eduardo mantiene Pytest de rutas críticas y Playwright E2E.
- Antes de mergear: lint + tests + build.
- Antes de evaluar: Docker + ejecutar smoke test.

13. Equipo, roles y tareas

El subject exige Product Owner, Project Manager/Scrum Master, Technical Lead/Architect y Developers. Con 5 personas se pueden combinar roles. Todos siguen siendo desarrolladores y deben contribuir tanto a Mandatory como a módulos.

Integrante	Rol técnico	Rol de proyecto
Florinda	Frontend 1 - Lead, Public & Admin	Product Owner + Developer
Lylia	Frontend 2 - Client & Integration	Developer
Ana	Backend 1 - API & Business Logic	Technical Lead/Architect + Developer
Daruny	Backend 2 - Data & Infrastructure	Data/Persistence Owner + Developer
Eduardo	Integration /QA /DevOps	QA/infraestructura + Developer

13.1 Florinda - Frontend Lead / Public & Admin

- Mantener la visión del producto y priorizar el backlog junto al equipo.
- Definir la estructura visual de Vue: layouts, navegación y criterios de UI compartidos.
- Desarrollar Landing Page, Privacy Policy, Terms of Service y zona pública.
- Crear y mantener componentes visuales base: Header, Sidebar, Cards, Modal y sistema de estilos.
- Desarrollar Dashboard Admin: KPIs, clientes/sites, mapa Leaflet y vistas principales.
- Integrar estados visuales, filtros y navegación de la zona Admin junto a Lylia.
- Supervisar responsive, UX y accesibilidad del frontend completo.
- Revisar contratos OpenAPI y Pull Requests que afecten a la experiencia Admin o componentes compartidos.

13.2 Lylia - Frontend Client & Integration

- Configurar Pinia, stores compartidos y estado global de autenticación, sensores y alertas.
- Configurar Axios, services/composables, interceptores y adaptadores Mock/API.
- Implementar Login, Registro, Logout, sesión y guards por rol en coordinación con Backend.
- Desarrollar sensores, alertas, tablas y filtros de la zona Admin como apoyo a Florinda.
- Construir el Dashboard Cliente completo: sensores propios, alertas, históricos y perfil.
- Implementar Chart.js, filtros temporales y conexión WebSocket cuando REST esté estable.
- Liderar Vitest/E2E de frontend y estados comunes de loading, error y empty state.

13.3 Ana - Backend API & Business Logic

- Definir y mantener la arquitectura FastAPI del monolito modular.
- Configurar main.py, routers, dependencias y manejo global de errores.
- Definir contratos Pydantic y mantener Swagger/OpenAPI como contrato con Frontend.
- Implementar endpoints REST y lógica de negocio de auth, users, sites, sensors y readings.
- Implementar autenticación, hash de contraseñas, sesión/token, permisos y aislamiento por organización.
- Implementar reglas de alertas y coordinar su persistencia con Daruny.
- Añadir WebSockets y broadcasting únicamente cuando el flujo REST sea estable.
- Crear y mantener pruebas Pytest de services, endpoints, permisos y errores críticos.

13.4 Daruny - Backend Data & Infrastructure

- Configurar PostgreSQL, conexión SQLAlchemy y estrategia de sesiones/transacciones.
- Diseñar modelos, relaciones, índices y restricciones del dominio.
- Configurar Alembic, migraciones y seeders reproducibles.
- Implementar repositories y consultas para readings, alerts, históricos y analytics.
- Crear el simulador Python con escenarios normal, low/high pressure y sensor offline.
- Mantener Dockerfiles, compose.yaml, redes, volúmenes y health checks.
- Configurar Nginx, HTTPS, variables de entorno y comunicación entre contenedores.
- Preparar backups básicos, smoke tests y dataset opcional de carga/paginación.

13.5 Eduardo - Integration / QA / DevOps

- Configurar Docker Compose base, redes, volúmenes y comunicación entre servicios.
- Configurar Nginx Gateway y HTTPS.
- Añadir health checks y smoke test básico.
- Apoyar la integración global y el entorno de evaluación, sin asumir lógica de negocio.
- Crear/mantener Pytest de rutas críticas y Playwright E2E de autenticación y navegación.

13.6 Trabajo conjunto obligatorio

Pareja	Trabajo conjunto
Florinda + Lylia	Componentes compartidos, mocks/API, responsive, UX.
Ana + Daruny	Datos, módulos FastAPI, OpenAPI, transacciones, reglas.
Frontend + Backend	Contratos JSON, endpoints, errores, filtros y WebSockets.
Eduardo + equipo	Pytest, E2E, Docker, Nginx, health/smoke e integración.
Todo el equipo	Demo, README y evaluación.

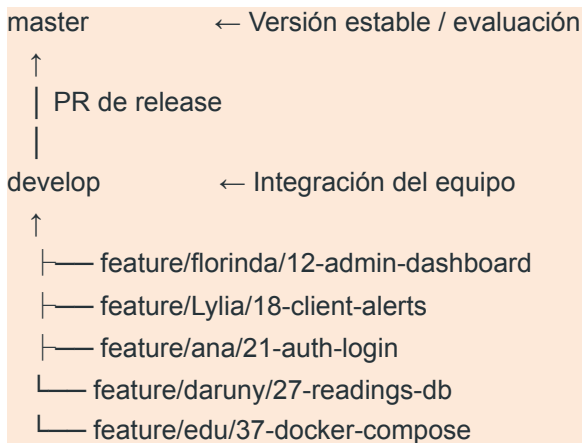
14. Git y gestión de ramas

Master para versiones estable.

Develop para integrar el trabajo del equipo de todas las issues.

No se trabaja en master ni develop.

Cada Issue se desarrolla en una rama nueva creada desde DEVELOP.



Flujo de trabajo

Issue -> Rama -> Desarrollo -> Pull Request -> Review + Actions -> Merge

1. Elegir una **Issue** y asignarla.
2. Actualizar develop y crear una rama nueva.
3. Desarrollar, hacer commits, abrir **Pull Request** hacia develop.
4. **GitHub Actions** ejecuta lint, tests y build.
5. Cuando develop sea estable → **PR de develop a master**.

Nombre de ramas

feature/<responsable>/<issue>-<descripcion>

fix/<responsable>/<issue>-<descripcion>

Ejemplo:

fix/daruny/35-database-connection

15. Plan de módulos: mínimo 14 puntos

Después de completar el core Mandatory, el proyecto necesita un mínimo de 14 puntos en módulos. La selección siguiente es coherente con AquaGuard y evita IA, blockchain o microservicios.

Módulo	Tipo	Pts	Aplicación
Framework frontend + backend	Major	2	Vue + FastAPI.
ORM	Minor	1	SQLAlchemy.
Real-time features	Major	2	WebSockets: lecturas/alertas entre clientes; conexión y desconexión.
Advanced permissions	Major	2	CRUD usuarios, roles y vistas/acciones por rol.
Organization system	Major	2	CRUD organizaciones, miembros y acciones dentro de organización.
Advanced analytics dashboard	Major	2	Gráficas interactivas, tiempo real, exportación, rangos y filtros.
Advanced search	Minor	1	Filtros, sorting y paginación.
Data export/import	Minor	1	CSV/JSON, importación validada y operaciones bulk simples.
Notification system	Minor	1	Notificaciones de creación/actualización/eliminación relevantes.

Total planificado: 14 puntos.

Importante

Durante evaluación solo cuentan módulos completos y demostrables. Si un módulo queda a medias, vale 0. Conviene cerrar 14 primero y, si queda margen, preparar 1-2 puntos de seguridad adicionales antes de pensar en bonus.

16. Bonus recomendados

El bonus solo se considera después de tener implementados los 14 puntos requeridos. El subject permite como máximo 5 puntos extra.

Idea	Encaje en subject	Dificultad	Recomendación
PWA + notificaciones push	PWA Minor = 1 punto; push añade valor pero no suma por sí solo.	Baja-media	Muy recomendable: instalable en móvil y útil para alertas.
Health/status + backups	DevOps Minor = 1 punto.	Baja-media	Recomendable y muy alineado con monitorización.
i18n: ES/EN/CA	Multiple languages Minor = 1 punto, exige 3 traducciones completas.	Baja-media	Fácil de demostrar si se prepara desde frontend.
Detección de anomalías	Custom Module of Choice Major = 2 puntos si es sustancial y está bien justificado.	Media	Buena opción: tendencia progresiva, ventanas temporales, no ML complejo.
SMS a número móvil	No aparece como módulo estándar; podría proponerse como custom Minor si aporta complejidad suficiente.	Media	Validar antes con evaluadores; depende de servicio externo/API.
Chat Admin-Cliente	El Major "User interaction" exige chat + perfiles + sistema de amigos.	Media-alta	No es un bonus "fácil" si queréis reclamar los 2 puntos completos.

16.1 Combinación bonus sugerida (hasta 5)

- Custom anomaly detection (2).
- PWA + web push (1).
- Health/status/backups (1).
- i18n ES/EN/CA (1).

Alternativa: sustituir i18n por SMS como Custom Minor únicamente si el equipo confirma previamente que el alcance será aceptable como módulo personalizado.

17. Plan máximo de 6 semanas

La planificación está orientada a tener integración real desde la primera semana. Los bonus se dejan para el final y solo entran si no comprometen los 14 puntos.

Semana	Frontend	Backend/Data/Infra	Resultado
1 - Base + vertical	Setup Vue, layout, mocks, vista sensores mínima.	FastAPI, DB v1, Docker, Nginx base, POST/GET readings, simulador mínimo.	Simulador -> API -> DB -> Vue funcionando.
2 - Mandatory Auth	Registro/login, guards, legal pages, responsive base.	Auth seguro, users/orgs, validación, HTTPS estable, migraciones.	Mandatory técnico principal cerrado.
3 - Producto MVP	Admin + Client dashboard, mapa, alertas, histórico.	Sensors, alerts, reglas, permisos, endpoints completos MVP.	Demo de producto usable.
4 - Módulos core	Realtime UI, roles, organización, search/paginación.	WebSockets, roles/orgs, query filters, concurrencia.	Mayor parte de los 14 puntos.
5 - Analytics + calidad	Chart.js, export/import UI, notificaciones, E2E.	Analytics, CSV/JSON, notification events, tests, optimización.	14 puntos funcionales y probados.
6 - Release + bonus controlado	Polish, a11y, Chrome, README/demo; PWA/i18n si cabe.	Security review, backups/status; anomalías si cabe.	Release candidate y bonus solo si core estable.

17.1 Ritmo semanal

- Una reunión semanal de 30 minutos - 1 hora (día a gestionar en equipo).
- Una integración frontend-backend obligatoria a mitad de semana.
- Demo el último día de semana.
- Cada issue tiene responsable, revisor y criterio de aceptación.